

Task 1

Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and System throughput under different buffer sizes.

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define ITEMS 100000

int buffer_size;
int *buffer;

int in = 0;
int out = 0;

sem_t empty;
sem_t full;
pthread_mutex_t mutex;

void *producer(void *arg)
{
    int i;

    for (i = 0; i < ITEMS; i++)
    {
        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        in = (in + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    int i;
    int item;

    for (i = 0; i < ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        item = buffer[out];
```

```

        out = (out + 1) % buffer_size;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
    }

    return NULL;
}

int main()

    pthread_t producer_thread;
    pthread_t consumer_thread;

    struct timespec start, end;

    printf("Enter buffer size: ");
    scanf("%d", &buffer_size);

    buffer = malloc(buffer_size * sizeof(int));

    if (buffer == NULL)
    {

```

```

        printf("Memory allocation failed\n");
        return 1;
    }

    sem_init(&empty, 0, buffer_size);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    clock_gettime(CLOCK_MONOTONIC, &start);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    clock_gettime(CLOCK_MONOTONIC, &end);

    double time_taken =
        (end.tv_sec - start.tv_sec) +
        (end.tv_nsec - start.tv_nsec) / 1000000000.0;

    printf("\nBuffer size: %d\n", buffer_size);
    printf("Items produced: %d\n", ITEMS);
    printf("Items consumed: %d\n", ITEMS);
    printf("Time taken: %.6f seconds\n", time_taken);

```

```

    printf("Throughput: %.2f items/second\n",
           ITEMS / time_taken);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    free(buffer);

    return 0;
}

```

Output:

```

geya@LenovoL480:~/OS/Practical/Week 12$ ./task1
Enter buffer size: 1

Buffer size: 1
Items produced: 100000
Items consumed: 100000
Time taken: 4.759909 seconds
Throughput: 21008.80 items/second
geya@LenovoL480:~/OS/Practical/Week 12$ ./task1
Enter buffer size: 5

Buffer size: 5
Items produced: 100000
Items consumed: 100000
Time taken: 1.259356 seconds
Throughput: 79405.68 items/second
geya@LenovoL480:~/OS/Practical/Week 12$ |

```

Task 2

Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Code:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resourceA = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resourceB = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
    printf("Thread 1: locking Resource A\n");
    pthread_mutex_lock(&resourceA);

    printf("Thread 1: Resource A locked\n");
    sleep(1);

    printf("Thread 1: waiting for Resource B\n");
    pthread_mutex_lock(&resourceB);

    printf("Thread 1: Resource B locked\n");

    pthread_mutex_unlock(&resourceB);
    pthread_mutex_unlock(&resourceA);

    return NULL;
}
```

```

void *thread2(void *arg)
{
    printf("Thread 2: locking Resource B\n");
    pthread_mutex_lock(&resourceB);

    printf("Thread 2: Resource B locked\n");
    sleep(1);

    printf("Thread 2: waiting for Resource A\n");
    pthread_mutex_lock(&resourceA);

    printf("Thread 2: Resource A locked\n");

    pthread_mutex_unlock(&resourceA);
    pthread_mutex_unlock(&resourceB);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Program completed\n");

    return 0;
}

```

Output:

```

geya@LenovoL480:~/OS/Practical/Week 12$ ./task2
Thread 1: locking Resource A
Thread 1: Resource A locked
Thread 2: locking Resource B
Thread 2: Resource B locked
Thread 2: waiting for Resource A
Thread 1: waiting for Resource B

```